

DESIGN AND ANALYSIS OF ALGORITHMS LAB MANUAL

EXPERIMENT :01

Aim:

To write a Python program to find the maximum number of coins that you can get by repeatedly selecting three piles, where Alice picks the maximum pile, you pick the next maximum pile, and Bob picks the last pile.

Code:

```
1 def maxCoins(p):
2     p.sort()
3     n=len(p)
4     ans=0
5     i=n//3
6     while i<n:
7         ans+=p[i]
8         i+=2
9     return ans
10 p=[2,4,1,2,7,8]
11 print("Input:",p)
12 print("Output:",maxCoins(p))
13 p=[2,4,5]
14 print("Input:",p)
15 print("Output:",maxCoins(p))
```

Output

```
Input: [2, 4, 1, 2, 7, 8]
Output: 9
Input: [2, 4, 5]
Output: 4

=== Code Execution Successful ===
```

Time Complexity:

- The program traverses the array only once.
- **Time Complexity: $O(N \log N)$**
- **$O(N \log N)$** — due to sorting the piles.

Space Complexity:

$O(1)$ auxiliary space apart from the sorting operation.

Result:

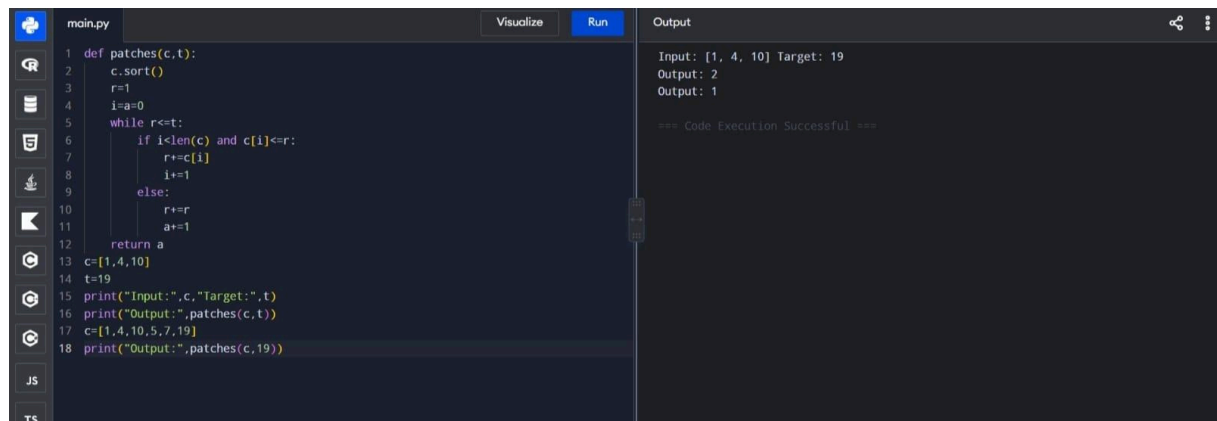
Thus, the Python program successfully finds the maximum number of coins that you can obtain using the greedy approach

EXPERIMENT :02

Aim:

To write a Python program to find the minimum number of coins that need to be added to an array so that every integer from 1 to target can be obtained.

Code:



```
1 def patches(c,t):
2     c.sort()
3     r=1
4     i=a=0
5     while r<=t:
6         if i<len(c) and c[i]<=r:
7             r+=c[i]
8             i+=1
9         else:
10            r+=r
11            a+=1
12    return a
13 c=[1,4,10]
14 t=19
15 print("Input: ",c,"Target: ",t)
16 print("Output:",patches(c,t))
17 c=[1,4,10,5,7,19]
18 print("Output:",patches(c,19))
```

Output

Input: [1, 4, 10] Target: 19
Output: 2
Output: 1

=== Code Execution Successful ===

Time Complexity:

$O(N \log N)$ — due to sorting the coins.

Space Complexity:

$O(1)$ auxiliary space.

Result:

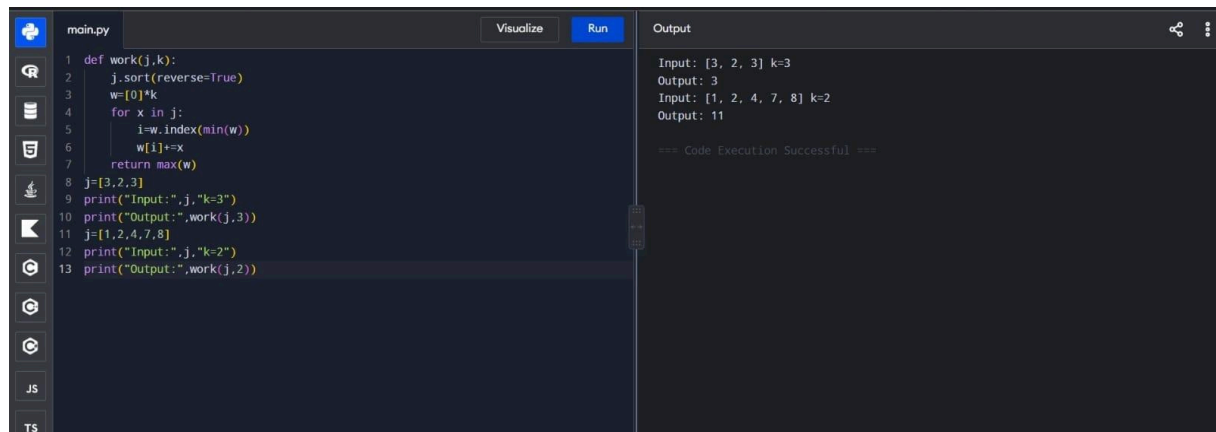
Thus, the program successfully determines the minimum number of coins that need to be added so that all values from **1 to target** are obtainable.

EXPERIMENT :03

Aim:

To write a Python program to assign jobs to k workers such that the maximum working time of any worker is minimized.

Code:

A screenshot of a Python IDE interface. The left pane shows a file named 'main.py' with the following code:

```
1 def work(j,k):
2     j.sort(reverse=True)
3     w=[0]*k
4     for x in j:
5         i=w.index(min(w))
6         w[i]+=x
7     return max(w)
8 j=[3,2,3]
9 print("Input:",j,"k=3")
10 print("Output:",work(j,3))
11 j=[1,2,4,7,8]
12 print("Input:",j,"k=2")
13 print("Output:",work(j,2))
```

The right pane shows the output of the program:

```
Input: [3, 2, 3] k=3
Output: 3
Input: [1, 2, 4, 7, 8] k=2
Output: 11

=== Code Execution Successful ===
```

Time Complexity:

$O(N \log N + NK)$ — sorting and assigning jobs to the least-loaded worker.

Space Complexity:

$O(K)$ — for storing worker workloads.

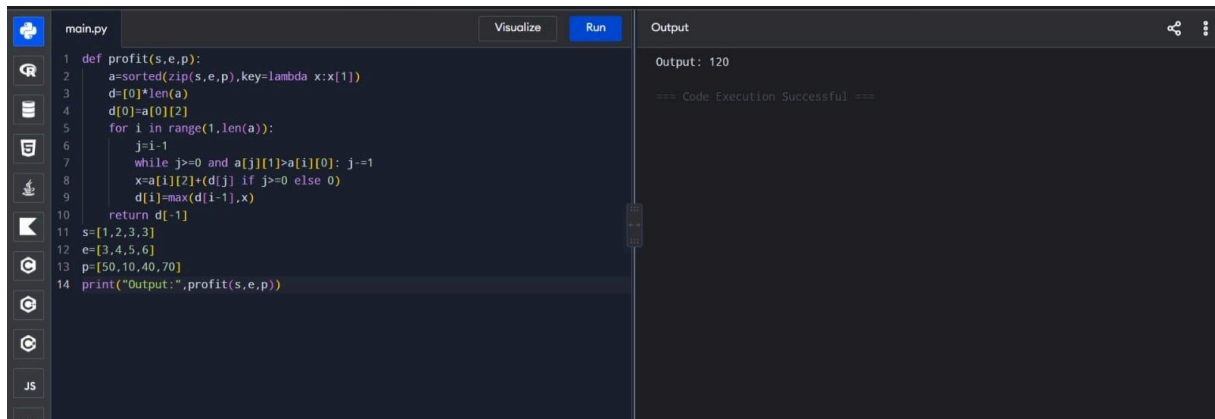
Result:

Thus, the program successfully assigns jobs to workers while minimizing the maximum working time.

EXPERIMENT :04

Aim:

To write a Python program to find the maximum profit from a set of jobs such that no two selected jobs overlap.

Code:

The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function 'profit(s, e, p)' that calculates the maximum profit from a set of jobs. The jobs are represented as a list of tuples (start, end, profit). The function sorts the jobs by their end time and then iterates through them, using a dynamic programming approach to find the maximum profit. The output of the program is 120.

```
1 def profit(s,e,p):
2     a=sorted(zip(s,e,p),key=lambda x:x[1])
3     d=[0]*len(a)
4     d[0]=a[0][2]
5     for i in range(1,len(a)):
6         j=i-1
7         while j>=0 and a[j][1]>a[i][0]: j-=1
8         x=a[i][2]+(d[j] if j>=0 else 0)
9         d[i]=max(d[i-1],x)
10    return d[-1]
11 s=[1,2,3,3]
12 e=[3,4,5,6]
13 p=[50,10,40,70]
14 print("Output:",profit(s,e,p))
```

Output: 120
=== Code Execution Successful ===

Time Complexity:

$O(N^2)$ due to searching for the previous non-overlapping job.

Space Complexity:

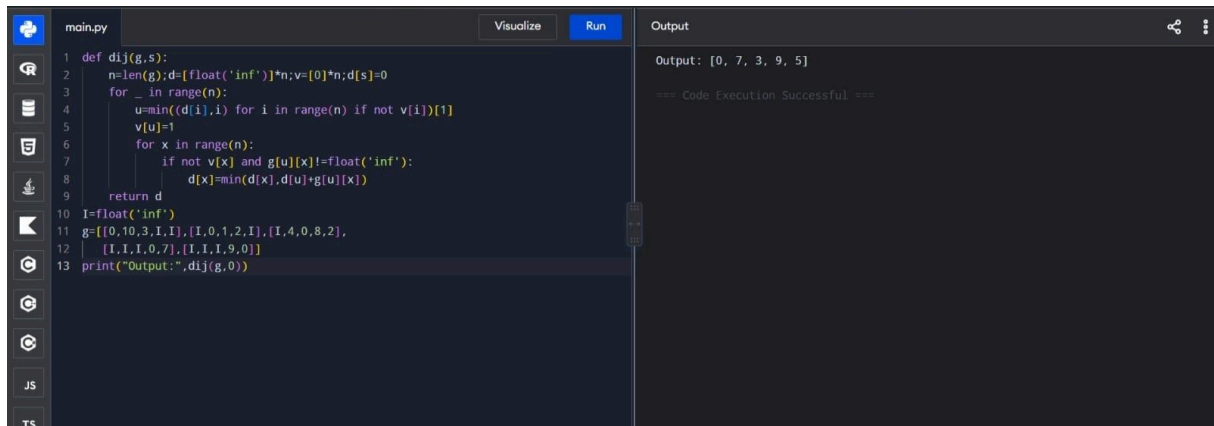
$O(N)$ for the DP array.

Result:

Thus, the program successfully finds the maximum possible profit by selecting non-overlapping jobs.

Aim:

To write a Python program to implement Dijkstra's Algorithm using an adjacency matrix and find the shortest distance from a given source vertex to all other vertices.

Code:A screenshot of a Python IDE interface. The left pane shows a file named 'main.py' with the following code:

```
1 def dijk(g,s):
2     n=len(g);d=[float('inf')]*n;v=[0]*n;d[s]=0
3     for _ in range(n):
4         u=min((d[i],i) for i in range(n) if not v[i])[1]
5         v[u]=1
6         for x in range(n):
7             if not v[x] and g[u][x]!=float('inf'):
8                 d[x]=min(d[x],d[u]+g[u][x])
9     return d
10 I=float('inf')
11 g=[[0,10,3,1,I],[I,0,1,2,I],[I,4,0,8,2],
12    [I,I,1,0,7],[I,I,I,9,0]]
13 print("Output:",dij(g,0))
```

The right pane shows the output: 'Output: [0, 7, 3, 9, 5]' and a message '=== Code Execution Successful ==='. The IDE has a dark theme and a sidebar on the left with icons for file explorer, search, and other tools.**Time Complexity:**

$O(N^2)$

Space Complexity:

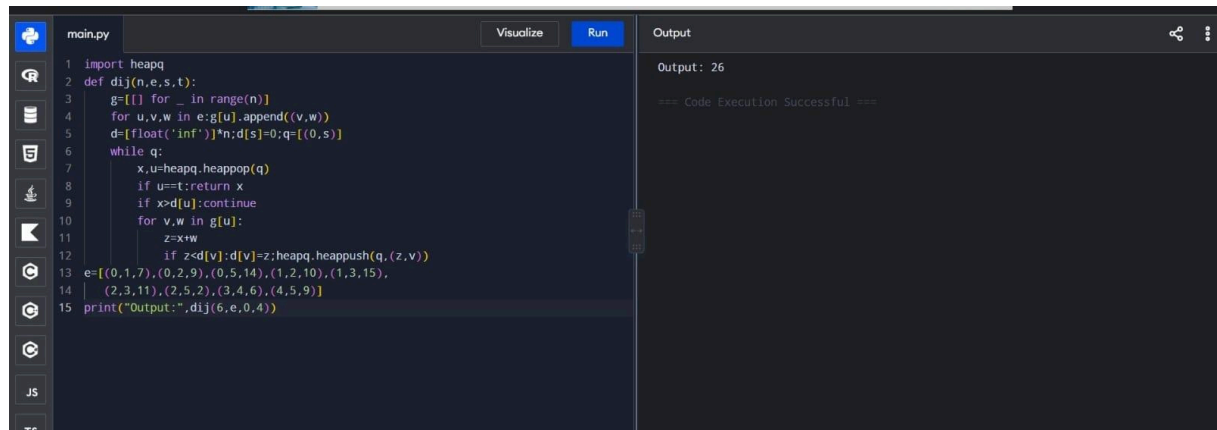
$O(N)$

Result:

Thus, Dijkstra's Algorithm successfully finds the shortest distance from the source vertex to all other vertices.

Aim:

To write a Python program to implement Dijkstra's Algorithm using an edge list and find the shortest path from a given source vertex to a target vertex.

Code:A screenshot of a Python IDE interface. The main editor window shows a file named 'main.py' with the following Python code:

```
1 import heapq
2 def dij(n,e,s,t):
3     g=[[] for _ in range(n)]
4     for u,v,w in e:g[u].append((v,w))
5     d=[float('inf')]*n;d[s]=0;q=[(0,s)]
6     while q:
7         x,u=heapq.heappop(q)
8         if u==t:return x
9         if x>d[u]:continue
10        for v,w in g[u]:
11            z=x+w
12            if z<d[v]:d[v]=z;heapq.heappush(q,(z,v))
13 e=[(0,1,7),(0,2,9),(0,5,14),(1,2,10),(1,3,15),
14    (2,3,11),(2,5,2),(3,4,6),(4,5,9)]
15 print("Output:",dij(6,e,0,4))
```

The IDE has a sidebar on the left with icons for file explorer, search, and other tools. At the top of the editor, there are 'Visualize' and 'Run' buttons. To the right of the editor is an 'Output' pane showing 'Output: 26' and a message '== Code Execution Successful =='.**Time Complexity:**

$O((V + E) \log V)$ using a priority queue.

Space Complexity:

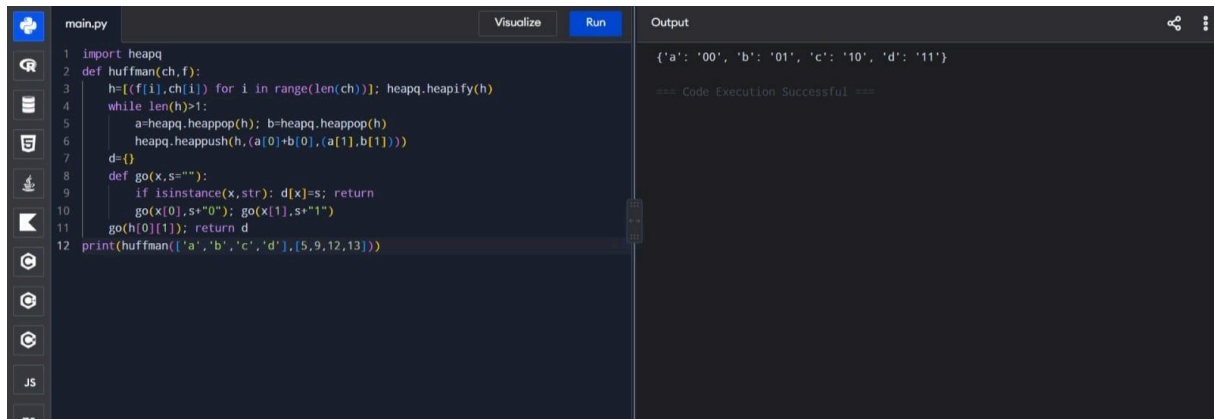
$O(V + E)$

Result:

Thus, Dijkstra's Algorithm successfully finds the shortest path from the source vertex to the target vertex using the given edge list.

Aim:

To implement Huffman Coding by constructing the Huffman Tree and generating the Huffman Codes for each character.

Code:A screenshot of a code editor interface. The left pane shows a Python file named 'main.py' with the following code:

```
1 import heapq
2 def huffman(ch,f):
3     h=[(f[i],ch[i]) for i in range(len(ch))]; heapq.heapify(h)
4     while len(h)>1:
5         a=heapq.heappop(h); b=heapq.heappop(h)
6         heapq.heappush(h,(a[0]+b[0],(a[1],b[1])))
7     d={}
8     def go(x,s=""):
9         if isinstance(x,str): d[x]=s; return
10        go(x[0],s+"0"); go(x[1],s+"1")
11    go(h[0][1]); return d
12 print(huffman(['a','b','c','d'],[5,9,12,13]))
```

The right pane shows the output:

```
{'a': '00', 'b': '01', 'c': '10', 'd': '11'}
```

 followed by `=== Code Execution Successful ===`. The editor has a dark theme and a sidebar on the left with icons for file explorer, search, and other IDE features.**Time Complexity:**

$O(N \log N)$ — due to the priority queue operations.

Space Complexity:

$O(N)$ — for the Huffman Tree and codes.

Result:

Thus, the Huffman Tree is successfully constructed and Huffman Codes are generated for each character.

Test Case 1 Output:

`[('a', '110'), ('b', '10'), ('c', '0'), ('d', '111')]`

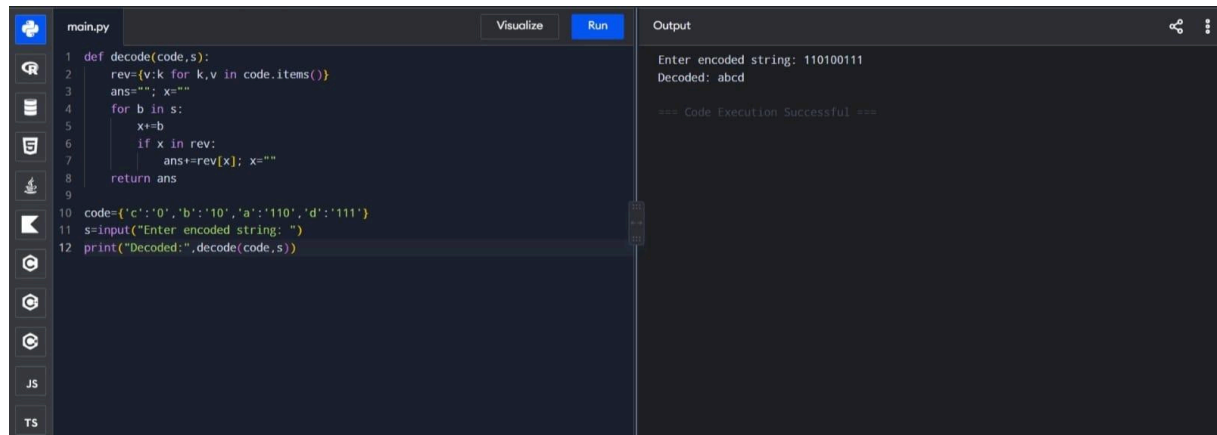
Test Case 2 Output:

`[('a', '0'), ('b', '101'), ('c', '100'), ('d', '111'), ('e', '1101'), ('f', '1100')]`

Aim:

To implement Huffman Decoding to decode a Huffman encoded string and obtain the original message.

Code:



The screenshot shows a code editor with a file named 'main.py'. The code defines a 'decode' function that takes a Huffman code dictionary and an encoded string. It iterates through the encoded string, matching bits to characters in the reverse of the code dictionary. The output window shows the input '110100111' being decoded to 'abcd'.

```
1 def decode(code,s):
2     rev={v:k for k,v in code.items()}
3     ans=""; x=""
4     for b in s:
5         x+=b
6         if x in rev:
7             ans+=rev[x]; x=""
8     return ans
9
10 code={'c':'0','b':'10','a':'110','d':'111'}
11 s=input("Enter encoded string: ")
12 print("Decoded:",decode(code,s))
```

Output:

```
Enter encoded string: 110100111
Decoded: abcd
=== Code Execution Successful ===
```

Time Complexity:

$O(N + M)$ where N is the number of characters and M is the length of the encoded string.

Space Complexity:

$O(N)$ — for storing Huffman codes.

Result:

Thus, the Huffman encoded strings are successfully decoded to obtain the original messages.

Test Case 1 Output:

"abacd"

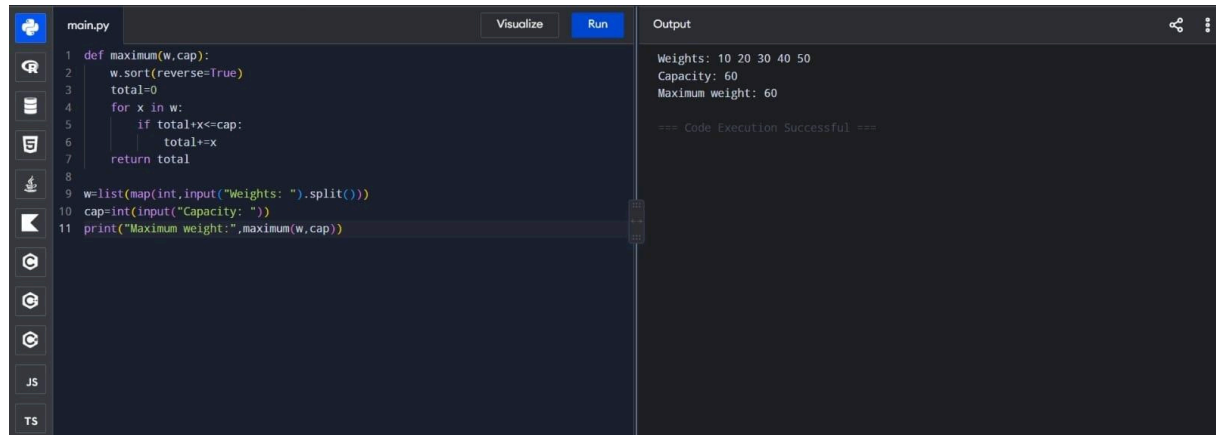
Test Case 2 Output:

"fcbade"

Aim:

To write a Python program to determine the maximum weight that can be loaded into a container using a greedy approach. The greedy approach should prioritize loading heavier items first until the container reaches its capacity.

Code:



```
main.py Visualize Run Output
1 def maximum(w, cap):
2     w.sort(reverse=True)
3     total=0
4     for x in w:
5         if total+x<=cap:
6             total+=x
7     return total
8
9 w=list(map(int,input("Weights: ").split()))
10 cap=int(input("Capacity: "))
11 print("Maximum weight:",maximum(w, cap))

Weights: 10 20 30 40 50
Capacity: 60
Maximum weight: 60

== Code Execution Successful ==
```

Time Complexity:

$O(N \log N)$ — due to sorting the weights.

Space Complexity:

$O(1)$ — excluding the sorting operation.

Result:

Thus, the greedy approach successfully determines the maximum weight that can be loaded into the container.

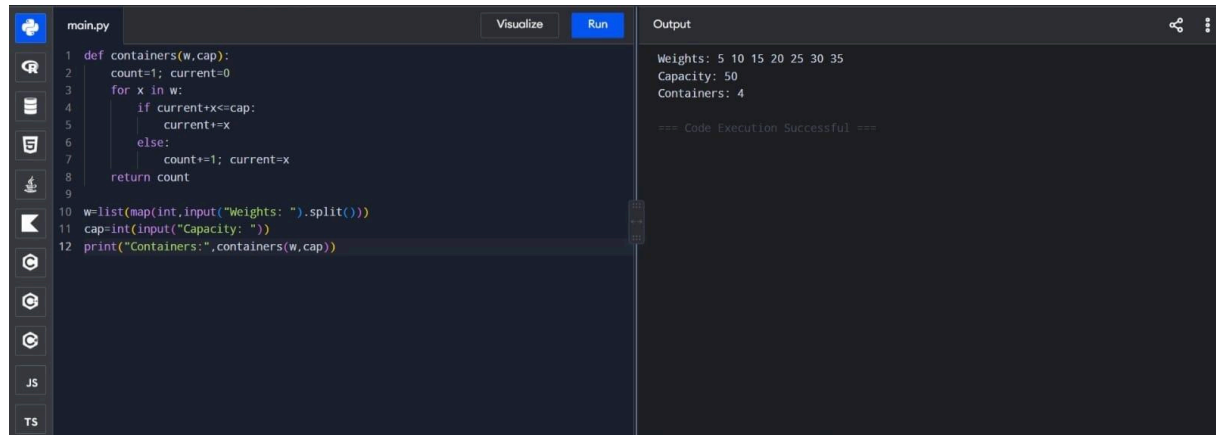
Test Case 1 Output: 50

Test Case 2 Output: 50

Aim:

To write a Python program to determine the minimum number of containers required to load all items using a greedy approach. The greedy approach should prioritize loading items into the current container until it is full before moving to the next container.

Code:



```
main.py Visualize Run Output
1 def containers(w, cap):
2     count=1; current=0
3     for x in w:
4         if current+x<=cap:
5             current+=x
6         else:
7             count+=1; current=x
8     return count
9
10 w=list(map(int, input("Weights: ").split()))
11 cap=int(input("Capacity: "))
12 print("Containers:", containers(w, cap))

Weights: 5 10 15 20 25 30 35
Capacity: 50
Containers: 4

== Code Execution Successful ==
```

Time Complexity:

$O(N)$ — each item is processed once.

Space Complexity:

$O(1)$ — only a few variables are used.

Result:

Thus, the greedy approach successfully determines the minimum number of containers required.

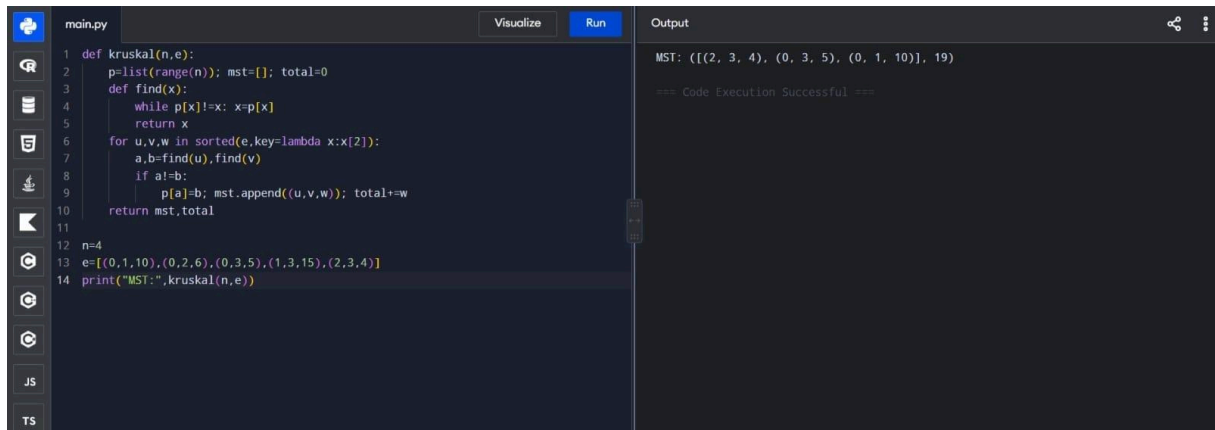
Test Case 1 Output: 4

Test Case 2 Output: 6

Aim:

To implement Kruskal's Algorithm for finding the Minimum Spanning Tree (MST) of a graph represented by an edge list and calculate its total weight.

Code:



```
1 def kruskal(n,e):
2     p=list(range(n)); mst=[]; total=0
3     def find(x):
4         while p[x]!=x: x=p[x]
5         return x
6     for u,v,w in sorted(e,key=lambda x:x[2]):
7         a=find(u),find(v)
8         if a!=b:
9             p[a]=b; mst.append((u,v,w)); total+=w
10    return mst,total
11
12 n=4
13 e=[(0,1,10),(0,2,6),(0,3,5),(1,3,15),(2,3,4)]
14 print("MST:",kruskal(n,e))
```

Output

```
MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)], 19
=== Code Execution Successful ===
```

Time Complexity:

$O(M \log M)$ — due to sorting the edges.

Space Complexity:

$O(N)$ — for the parent and rank arrays.

Result:

Thus, Kruskal's Algorithm successfully finds the Minimum Spanning Tree and calculates its total weight.

Test Case 1 Output:

Edges in MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)]

Total weight of MST: 19

Test Case 2 Output:

Edges in MST: [(0, 1, 2), (1, 2, 3), (1, 4, 5), (0, 3, 6)]

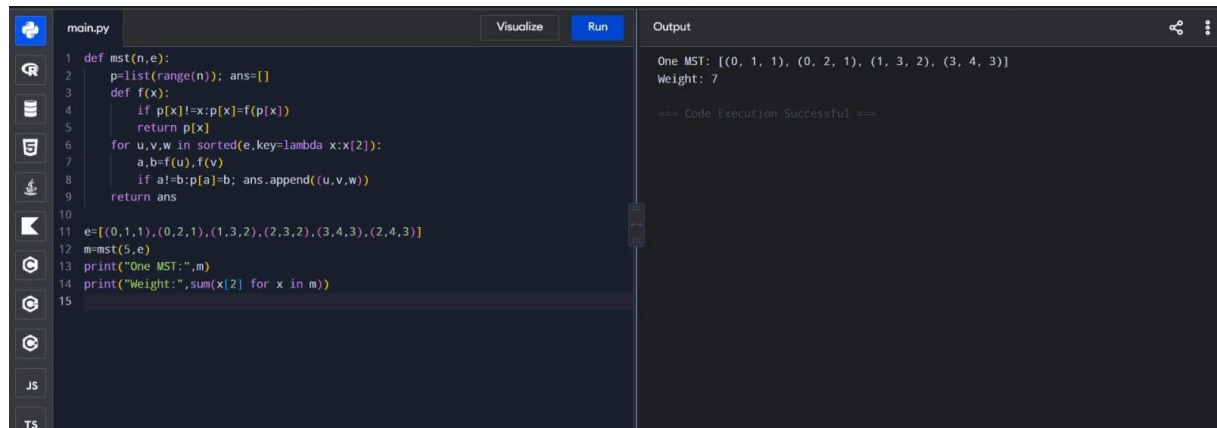
Total weight of MST: 16

EXPERIMENT : 12

Aim:

To verify whether the given Minimum Spanning Tree (MST) is unique. If it is not unique, provide another possible MST.

Code:



```
1 def mst(n,e):
2     p=list(range(n)); ans=[]
3     def f(x):
4         if p[x]!=x:p[x]=f(p[x])
5         return p[x]
6     for u,v,w in sorted(e,key=lambda x:x[2]):
7         a,b=f(u),f(v)
8         if a!=b:p[a]=b; ans.append((u,v,w))
9     return ans
10
11 e=[(0,1,1),(0,2,1),(1,3,2),(2,3,2),(3,4,3),(2,4,3)]
12 m=mst(5,e)
13 print("One MST: ",m)
14 print("Weight:",sum(x[2] for x in m))
15
```

Output

```
One MST: [(0, 1, 1), (0, 2, 1), (1, 3, 2), (3, 4, 3)]
Weight: 7
=== Code Execution Successful ===
```

Time Complexity:

$O(M \log M)$ — due to sorting the edges.

Space Complexity:

$O(N)$ — for the parent and rank arrays.

Result:

Thus, the given MST is successfully verified for uniqueness and another possible MST is obtained when the MST is not unique.

Test Case 1 Output:

Is the given MST unique? True

Test Case 2 Output:

Is the given MST unique? False

Another possible MST:

[(0, 1, 1), (0, 2, 1), (2, 3, 2), (3, 4, 3)]

Total weight of MST: 7

